OS Unit 4 Orange Problem

Name: Arvind Ajay Bharadwaj

SRN: PES1UG24AM052

Semester: 4

Section: A

Phase 1A

```
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$
```

Phase 1B

```
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$ xxd .pes/objects/^C
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20

00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
arvind@arvind:~/phase-1/PES1UG24AM052-pes-vcs$
```

Phase 2A

```
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$
```

Phase 2B

```
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$
```

Phase 3A

```
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ echo "hello" > file1.txt
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ echo "world" > file2.txt
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ ./pes add file1.txt file2.txt
ADD: file1.txt
Segmentation fault (core dumped)
arvind@arvind:~/os/PES1UG24AM052-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  commit.c
  untracked:  file2.txt
  untracked:  a.txt
  untracked:  tree.c
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  file1.txt
  untracked:  object.h
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  .gitignore
  untracked:  object.c
  untracked:  index.c
  untracked:  index.h
```

Phase 4A

```
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$ ./pes log
commit 1a33bed98dfb28fec666536e8430ecd6b0c04573554a40e881010d852726f02b
Author: Arvind Ajay Bharadwaj <PES1UG24AM052>
Date:   1776761889

    Add farewell

commit 791c76beb913a68aa88f691046b388fe491cac726bed5dc53c72079696dade16
Author: Arvind Ajay Bharadwaj <PES1UG24AM052>
Date:   1776761880

    Add world

commit 9595e7c8b940d0671350027e0fb90f9afa5298a3ac724c887bf817fa7b00f6af
Author: Arvind Ajay Bharadwaj <PES1UG24AM052>
Date:   1776761859

    Initial commit
```

Phase 4B

```
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/1a/33bed98dfb28fec666536e8430ecd6b0c04573554a40e881010d852726f02b
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/79/1c76beb913a68aa88f691046b388fe491cac726bed5dc53c72079696dade16
.pes/objects/95/95e7c8b940d0671350027e0fb90f9afa5298a3ac724c887bf817fa7b00f6af
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$
```

Phase 4C

```
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$ cat .pes/refs/heads/main
1a33bed98dfb28fec666536e8430ecd6b0c04573554a40e881010d852726f02b
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
arvind@arvind:~/os-orange/PES1UG24AM052-pes-vcs$
```

Phase 5

Q5.1

Checkout updates .pes/HEAD to point to the target branch, reads the commit hash from .pes/refs/heads/<branch>, loads its tree, and rewrites the working directory to match that tree. It also updates the index to reflect the new snapshot. The complexity comes from safely handling file overwrites, deletions, and directory structure changes while avoiding loss of uncommitted work.

Q5.2

For each tracked file, compare the working directory state with the index by recomputing the hash. If a file is modified relative to the index and also differs between the current tree and target tree, checkout must refuse. This detects conflicts by comparing working directory vs index and index vs target tree.

Q5.3

In detached HEAD, commits are created normally but no branch points to them, so they become unreachable once you move away. The user can recover them if they know the commit hash by creating a new branch pointing to it, or by checking it out again before it is garbage collected.

Phase 6

Q6.1

Start from all branch heads and traverse commits, trees, and blobs using DFS or BFS, marking all reachable object hashes in a hash set. Then scan the object store and delete any object not in the reachable set. With 100k commits and shared history, traversal is roughly 100k commits plus their trees and blobs, not multiplied by branches.

Q6.2

If GC runs while a commit is being created, it may delete objects (like blobs or trees) that are written but not yet referenced by any commit. The commit then ends up pointing to missing objects. Git avoids this by writing objects before updating refs, delaying deletion of recent objects, and using safeguards like timestamps and locks to prevent such races.